moon-slice — 项目申报书（一页）

> 仓库：https://github.com/cs6210501-lab/moon-slice | 在线演示：https://cs6210501-lab.github.io/moon-slice/

1. 项目名称

moon-slice — MoonBit 程序切片与依赖图工作台

2. 项目简介

moon-slice 是一个面向 MoonBit 的开发者工具。用户选中一个变量或一行语句， 工具基于 MoonBit 官方 AST 构建控制流图（CFG）、定义-使用链（def-use）、 控制依赖图（CDG）与程序依赖图（PDG），输出向后切片（这个值从哪来） 和向前切片（改这个值会影响谁），并给出源码高亮、JSON、DOT 和自包含 HTML 报告。 报告内联 SVG 依赖图（按源码顺序分层，数据边实线、控制边虚线），不依赖网络； 另有 `path --from A --to B` 数据流路径查询，用于 source→sink 的 taint-lite 审计。 它解决的是“值从哪来、改动影响谁、哪段代码是死的/不可达”这三个日常问题。

3. 项目方向与通用性说明

方向：开发者工具 / 工具库。 通用性：切片与 PDG 是语言无关的程序分析基础；本项目把 MoonBit 官方 AST 归一化为稳定的 `SNode` IR，后续可扩展到过程间切片、污点分析、 测试影响分析和 VS Code 插件，不局限于某一个示例程序。

4. 至少 3 个完整使用场景

1. **调试**：某个变量在函数末尾变成 `NaN` 或错误值。对目标行做向后切片， 工具列出所有可能影响该值的赋值、分支条件和调用，快速缩小排查范围。
2. **重构 / 影响分析**：准备修改一个变量或函数的返回值。先做向前切片， 工具列出所有受影响的语句与分支，避免漏改调用方或破坏边界条件。
3. **代码审查 / 安全审计**：`dead` 找出无使用的赋值与 `return` 后的不可达代码； `path --from A --to B` 沿 PDG 找 source→sink 的最短数据流路径，辅助审计敏感数据流。

5. 拟实现的核心功能

- MoonBit 官方 parser 适配与语句级 IR（`SNode`、`VarId`、`Loc`）。
- 覆盖自由函数与 `impl T with fn` 方法 / view 方法体；CFG：顺序、if/else、guard/else、while、for（含 C 风格三段式与 for-in）、match、return/break/continue。
- def-use：reaching definitions；对字段/数组写做保守处理。
- CDG：结构化控制依赖；return/break/continue 保守连接。
- PDG：数据依赖 + 控制依赖。
- 切片：backward / forward，输出节点集合与源码行号。
- 报告：JSON、DOT、自包含 HTML（内联 SVG 依赖图 + 源码高亮 + 切片节点表）。
- 查询：`dead`（无使用定义 + 不可达代码）、`path`（source→sink 数据流路径）。
- 包级存活分析：`unused --dir D` 找出整个包内已无人调用的私有函数（以目录为包）。排除 `main`/`init`、`pub` API、方法与仅有声明的桩；名字在包内任何位置出现过即不报，命中即“全包只出现一次”。
- CLI：`slice --interprocedural`、`graph`、`callgraph`、`dead`、`path`、`html`，以及 `dump-*` 调试命令。
- 过程间：基于 `Program.calls` 的返回值 cone 展开（裸调用语句展开整个函数体）；调用图覆盖 let/assign/return、嵌套调用、条件位置与限定调用 `T::f(x)`，保留递归自环。
- 包级切片：`slice --dir D --file F --line N --interprocedural` 把被调函数追到 D 下任意文件（同文件优先），结果按文件分组；裸调用展开整个函数体，有返回值的调用只展开返回值 cone。
- 反向查询：`callers --dir D --name F` 列出包内所有调用点及所属函数（`--json` 含 caller 字段），在 moonbitlang/x 的 time 包上找出 10 个 `Duration::parse_str` 调用点。
- 改动影响分析：`review --diff`（配 `tools/review.sh`）对 git diff 的每个改动行做向前切片，汇总受影响行与函数。
- 仓库级死代码扫描：`dead --dir D` 一次进程分析整棵源码树（跳过 `.git/_build/target/.mooncakes/node_modules`），解析失败按文件报告且不中断其他文件。
- CI 集成：`dead --format sarif` 输出 SARIF 2.1.0（dead-store / unreachable-code 两条规则、artifact location、partialFingerprints）； `--write-baseline/--baseline` 记录并抑制存量； `--fail-on-findings` 仅对新增问题退出 1。
- GitHub Action： `action.yml` 封装安装/构建/执行， `templates/review-workflow.yml` 同时接入 PR 影响面报告与 code-scanning 上传。
- 真实仓库验证：`dead --dir` 在 moonbitlang/async（368 文件 / 50,388 行）与 moonbitlang/x（166 文件 / 37,595 行）上 100% 解析、0 死代码命中；批量扫描分别约 1.2 s / 1.0 s。自身仓库 7 处命中全部人工确认为真。
- 包级存活验证：`unused --dir` 在 async、x、core 合计 1987 个文件上 0 误报；core 首轮命中的 4 个全部带官方 `#warnings("-unused_value")` 标注（故意不调用），分析已尊重该标注。
- 真实提交验证：对 moonbitlang/x 提交 `4705764`（15 个改动行）做 review，正确报告 duration.mbt 影响 6 个函数/36 行、period.mbt 影响 3 个函数/4 行、测试文件 0 行。
- 测试：294 个测试，含 parser smoke、CFG 精确边、def-use、切片节点集合、 10 个示例的端到端断言、路径查询与不可达回归； `moon check` 0 warning。
- 精准度：独立预言器按"定义到使用存在一条不被重定义阻断的路径"重新推导 reaching definitions，与工作列表算法逐对比较；另有种子化属性测试，每轮生成 40 个程序检查结构不变量、预言器一致性、切片闭包与过程间超集性。
- 控制依赖双模式：默认结构化 CDG（不会漏报）； `--precise` 用后支配计算精确 控制依赖（消除 region→region 伪边与循环头/出口过度近似）。
- 性能：32000 条语句的直线函数， `dead` 0.80 s、 `graph` 0.74 s、 `slice` 1.55 s （native release， `bench/bench.mjs` 可复现）。
- 工程化：GitHub Actions CI、可复现的 32000 语句性能基准、零网络依赖报告。
- 浏览器 playground：同一管线编译到 JavaScript；点击行号即切片，行号槽标出切片/目标行，内置 5 个示例与 Copy link； `web/presets.mjs` 逐示例逐方向用 JS 引擎核对，数字与 CLI 一致。

6. 原创 / 移植说明

原创项目。算法思想参考 Weiser（1981）的程序切片与 Ferrante 等（1987）的 PDG 构造；不复制任何现有实现代码。使用 MoonBit 官方 `moonbitlang/parser` （Apache-2.0）作为解析前端。项目许可证：Apache-2.0。

7. GitHub 仓库

`https://github.com/cs6210501-lab/moon-slice`

仓库包含不少于 10 个有效 commits，保留连续开发记录、测试与 README。

8. 预期验收

- `./dev.sh check` 0 errors / 0 warnings； `./dev.sh test` 294/294 通过。
- `./dev.sh run cmd/main -- slice --file examples/04_while.mbt --line 7 --direction backward` 输出 `4,5,6,7,8`；forward 输出 `7,10`。
- `./dev.sh run cmd/main -- path --file examples/04_while.mbt --from i --to total` 输出 `n1 L4 -> n5 L7`。
- `./dev.sh run cmd/main -- slice --file bench/callgraph.mbt --line 12 --direction backward --interprocedural` 输出 `2,6,7,8,12`。
- `./dev.sh run cmd/main -- html --file examples/04_while.mbt --line 7 --out report.html` 生成可打开的自包含 HTML；渲染截图见 `docs/report.png`。
- `./dev.sh run cmd/main -- dead --dir examples` 输出 7 行； `--format sarif --out x.sarif` 生成可被 code scanning 解析的 SARIF 2.1.0。
- README 提供可复现 quickstart、架构说明、支持子集与诚实的局限性。